



Gradient Boosting (Scikit-Learn)

Классическая реализация градиентного бустинга в библиотеке Scikit-Learn представляет собой базовый алгоритм, описанный Джеромом Фридманом. Это стандартный, ванильный бустинг без продвинутых оптимизаций.

Алгоритм строит деревья строго последовательно, минимизируя функцию потерь с помощью шагов по антиградиенту. Для поиска оптимального разбиения в узле дерева алгоритм честно перебирает все возможные значения признаков для всех объектов.

Главный недостаток этой реализации — **низкая скорость обучения** на больших объемах данных. Здесь нет распараллеливания при построении одного дерева, нет встроенной работы с пропущенными значениями и нет нативной поддержки категориальных признаков. Для современного продакшена эта реализация считается устаревшей.



XGBoost

Библиотека, которая произвела революцию в машинном обучении и долгое время доминировала на платформе Kaggle. Ее главное отличие заложено в математике: алгоритм использует не только градиент, но и **гессиан**. Это позволяет делать шаги оптимизации более точными и быстрыми благодаря разложению в ряд Тейлора.

Второй важнейшей инновацией стала **встроенная регуляризация**. XGBoost штрафует модель за слишком сложные деревья и большие веса в листьях, это снижает риск переобучения.

Кроме того, алгоритм оптимизирован аппаратно. Он использует приближенные алгоритмы поиска сплитов на основе гистограмм, что позволяет не перебирать все точки, а также умеет эффективно работать с кэшем процессора и обучаться вне оперативной памяти для гигантских датасетов.



LightGBM

Разработка компании Microsoft, созданная с фокусом на максимальную скорость и низкое потребление памяти. Главное структурное отличие LightGBM заключается в подходе к росту дерева. В то время как классические бустинги растут по уровням (Level-wise), эта библиотека использует стратегию **роста по листьям (Leaf-wise)**.

Это означает, что алгоритм всегда выбирает для разбиения тот лист, который даст максимальное снижение функции потерь, игнорируя симметрию дерева. Это позволяет сходиться к оптимуму гораздо быстрее, хотя и повышает риск переобучения на малых датасетах.

Высокая скорость достигается за счет двух уникальных алгоритмов. **GOSS** отбрасывает объекты с маленькими градиентами (те, на которых модель уже не ошибается), фокусируясь только на сложных примерах. А алгоритм **EFB** объединяет разреженные, взаимоисключающие признаки в один, существенно сокращая количество столбцов без потери качества.



CatBoost

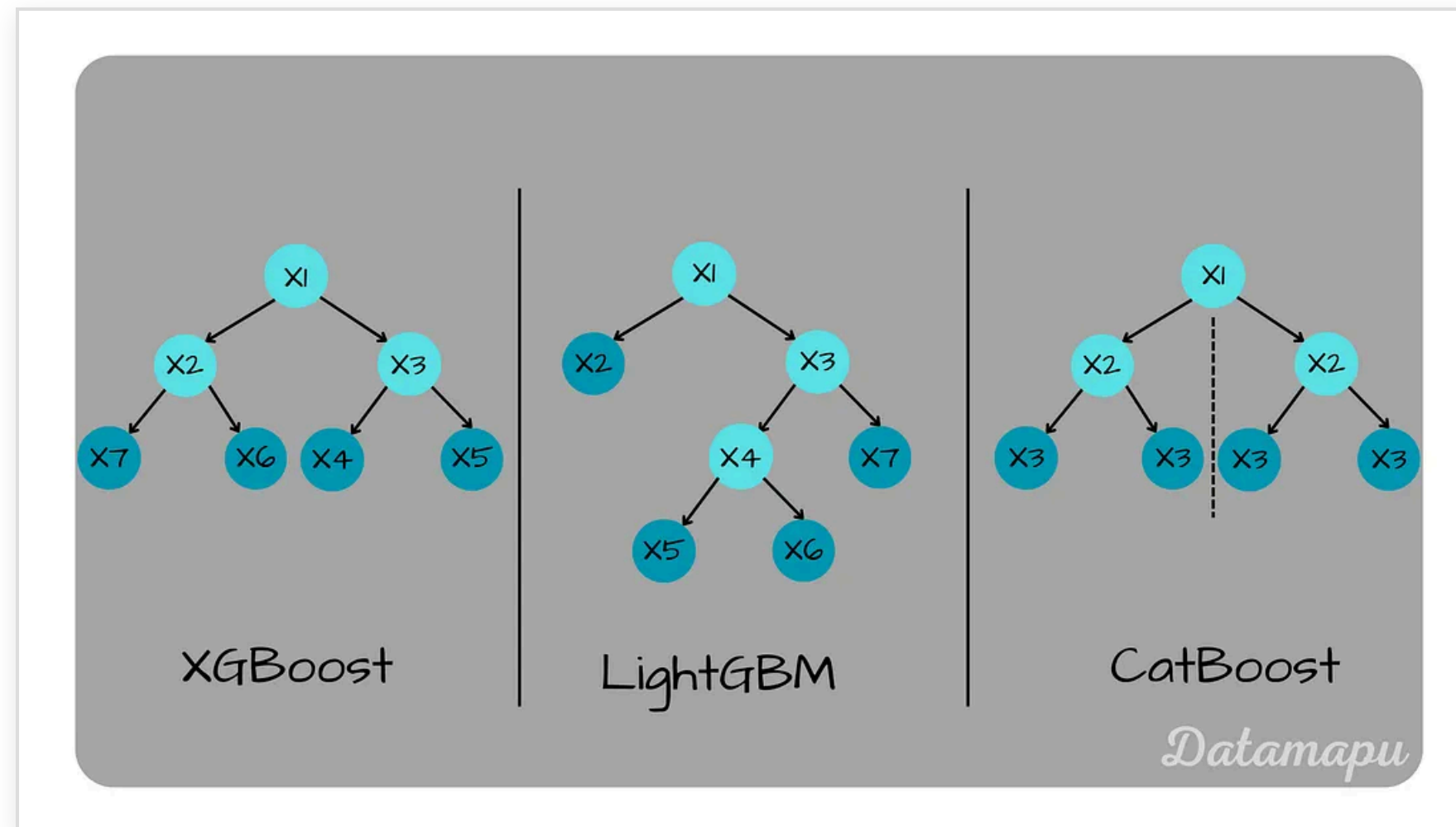
CatBoost проектировался для работы с категориальными признаками без предварительного One-Hot Encoding. Он применяет собственную технику **Ordered Target Encoding**. Базовая идея Target Encoding — заменить категорию (например, город «Москва») на среднее значение целевой переменной по этой категории. Чтобы при этом избежать утечки данных (когда ответ объекта влияет на его же признак), CatBoost вводит искусственное «время». Он случайно перемешивает датасет и, вычисляя среднее для конкретной строки, использует ответы только тех объектов, которые находятся строго «в прошлом» (выше по списку).

Этот же принцип историчности решает фундаментальную проблему **упорядоченного бустинга**. В классическом бустинге модель оценивает свою ошибку на объекте, хотя она уже видела этот объект при обучении. Это вызывает переобучение. CatBoost строит процесс иначе: чтобы получить честный градиент для объекта, алгоритм использует виртуальную модель, которая была обучена исключительно на объектах из его «прошлого». На практике CatBoost поддерживает сразу несколько случайных перестановок обучающей выборки, чтобы сделать этот процесс максимально надежным.

Структурно CatBoost использует **симметричные деревья**. Это означает, что на каждом уровне дерева для абсолютно всех узлов выбирается только один признак и один порог разбиения (например, весь второй уровень будет применять сплит «возраст > 30»). Такая жесткая архитектура выступает мощнейшим регуляризатором, отсекая шум. Кроме того, она превращает логику дерева в простейший бинарный индекс, делая процесс инференса самым быстрым среди всех библиотек при работе на CPU.



Красивая картинка



- **XGBoost:** Растет по уровням, сохраняя симметрию. Баланс между скоростью и точностью.
- **LightGBM :** Выбирает лист с наибольшим приростом. Максимальная точность, высокая скорость, риск переобучения на малых данных.
- **CatBoost:** Глобальные сплиты для уровня. Максимальная защита от переобучения и высочайшая скорость инференса.